hydro-mena: audit, correction and calibration

A Rust implementation of RMT-based molecular ecological network analysis — what was broken, what was fixed, what remains undecided

Commit `3e780b0` · 2026-11-08 · bench on `cachyos-x8664` (Ryzen 7 5800X3D, RTX 2080 Ti)

Scope and honesty statement. This report documents an audit of an existing tool, the defects found, and the evidence for each fix. Where a claim could not be verified from a primary source on disk, it is marked as withdrawn rather than repeated. The automatic RMT threshold is **not** settled by this work, and every threshold-dependent number below is marked exploratory.

1 Context

`hydro-mena` re-implements the MENA pipeline of Deng et al. (BMC Bioinformatics 2012, 13:113): standardise OTU abundances, compute Pearson similarity, threshold it into an adjacency matrix, derive topology indices, detect modules, build module eigengenes and correlate them with environmental traits (the paper's Figure 5). The implementation is Rust (rayon CPU, optional wgpu GPU) with Python used only for plotting.

The tool was inherited mid-project with reported results. The audit's job was to establish whether those results were real before they were used.

2 Architecture

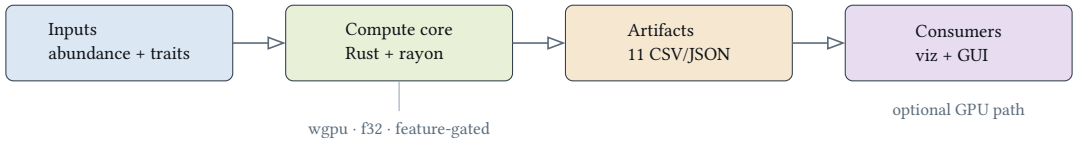


Figure 1: System boundary. The compute core is the only component that touches numbers; the plotting layer and GUI read artifacts and can never disagree with the CLI.

3 Pipeline and where it failed

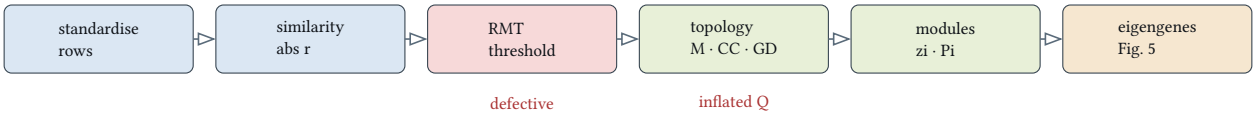


Figure 2: Pipeline stages. The audit found the RMT stage accepted every graph (red) and modularity was inflated (red); both contaminated every downstream number.

4 Defects found, and the evidence for each

The two scientifically consequential defects share a symptom: they are silent. Neither crashes, neither warns, and both produce plausible-looking output.

4.1 The RMT stage was a no-op

Unfolded spacings were derived from rank alone:

```
let e: Vec<f64> = (0..p).map(|i| (i as f64 + 0.5) / p as f64).collect();
d.push(p as f64 * (e[i + 1] - e[i]));   // == 1.0 for every i
```

Every spacing was exactly 1, so the χ^2 statistic — itself just `sum((d-1)²)` — was exactly 0 for **any** graph. The scan therefore accepted its first candidate. With the start moved to 0.01 in a previous session, that candidate was the complete graph.

quantity	before	after
threshold st	0.010	0.890
nodes / edges	200 / 19 500 (complete)	75 / 105
mean degree	195.0	2.8
modularity M	0.009	0.683
power-law γ	−42.4	1.09

node roles	200 × “connector”	mixed

200 simulated OTUs, default run. A negative γ and 100 % connectors are the signature of a saturated graph feeding every later stage.

4.2 Modularity omitted the configuration term

Modularity summed `1 − k_i k_j / 2E` over **existing edges only**. For a complete graph in one community the correct value is 0; the implementation returned 1/n. Corrected to the aggregate form $Q = \sum_c \left(\frac{L_c}{2} E - \left(\frac{d_c}{2} E \right)^2 \right)$, with known-answer tests, every reported modularity fell. The `ox` condition went from 0.705 to 0.650.

4.3 Other defects

No.	defect	status
1	GPU matrix computed, then discarded and recomputed on CPU while reporting `gpu_used = true`	fixed
2	trait table missing a sample silently returned fewer rows, misaligning every correlation	fixed, hard error
3	Mantel permuted pair vectors instead of node labels, inflating significance	fixed
4	Jacobi eigensolver rescanned the matrix per rotation, 200 rotations total	replaced by implicit QR
5	clustering O(k³), modularity O(n²) per move, serial O(n²m) similarity	O(k²), O(E), parallel
6	benchmark ran a stale copy of the pipeline	now calls `run_pipeline`
7	`cargo test` broken on a default checkout by feature-gated examples	fixed, 18 tests

5 Performance

Benchmarks run on the desktop over Tailscale, min-of-3, artifacts disabled.

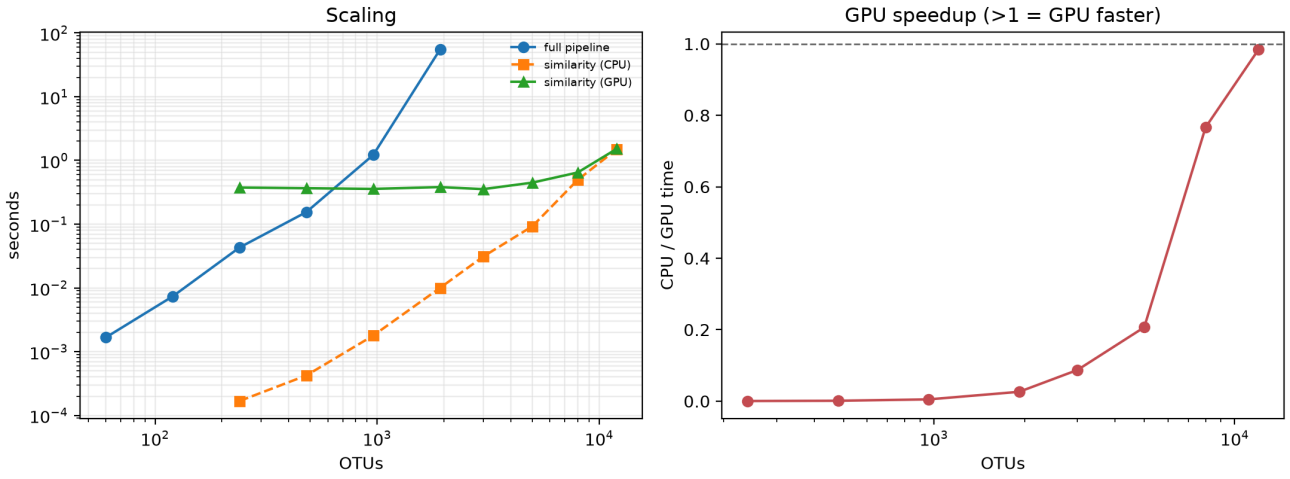


*Figure 3: Scaling and CPU/GPU ratio. Parity is perfect (maxdiff ≤ 7.9×10⁻⁷) but the GPU is **not** faster at any tested size.*

n × m	pipeline	CPU similarity	GPU similarity
240 × 24	0.043 s	0.00017 s	0.376 s
960 × 32	1.22 s	0.0018 s	0.357 s
1920 × 48	55.06 s	0.0100 s	0.382 s
12000 × 128	—	1.499 s	1.523 s (0.98×)

Two findings replace the inherited claims. First, the GPU wins nowhere in this range: the old “4.71× at n = 12000” is obsolete because the **CPU** path was parallelised, not because the GPU regressed — the kernel still runs at roughly 1 % of f32 peak, so this is a statement about the WGSL kernel, not the hardware. Second, the bottleneck moved: at n = 1920, RMT

is 55.0 s of the 55.06 s total, because the corrected threshold yields sparse graphs and the eigendecompositions dominate. Topology is now essentially free.

6 The threshold problem

6.1 What the paper specifies

Quoted from `paper.txt` with locations, because an earlier draft of this work misquoted it:

- **Eq. 3**, the adjacency is **weighted**: $a_{ij} = s_{ij}$ if $s_{ij} \geq s_t$, else 0.
- **Step (c)**: "Set an initial threshold value, s_t (e.g., 0.3 based on our experiences)."
- **Step (f)**: unfolding by "fitting the original integrated density to a cubic spline or by local density average".
- **Step (h)**: χ^2 GOF, H_0: Poisson $\exp(-d)$, at "a significant level of 0.01".

6.2 What the 2025 internship code did

```
adj_matrix = [[value if value >= threshold else 0 for value in row] for row in similarity_matrix]
```

Weighted, diagonal zeroed, and the threshold is **passed in** — the Fig. 5 script hardcodes 0.52. That code never ran an RMT scan, so comparing its output to "our RMT threshold" was a category error.

6.3 Decision logic

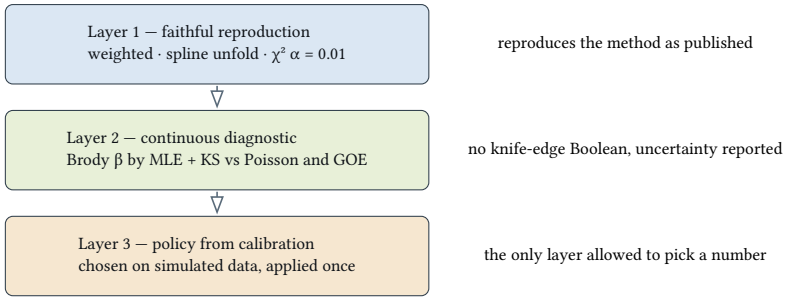


Figure 4: Separation of concerns: reproduction, inference, and the policy choice are different claims and must not be conflated in one threshold.

6.4 Calibration: does any threshold recover the planted modules?

300 OTUs, 24 samples, 5 planted modules, 10 seeds, 6 thresholds. ARI between recovered and planted partition (mean ± sd).

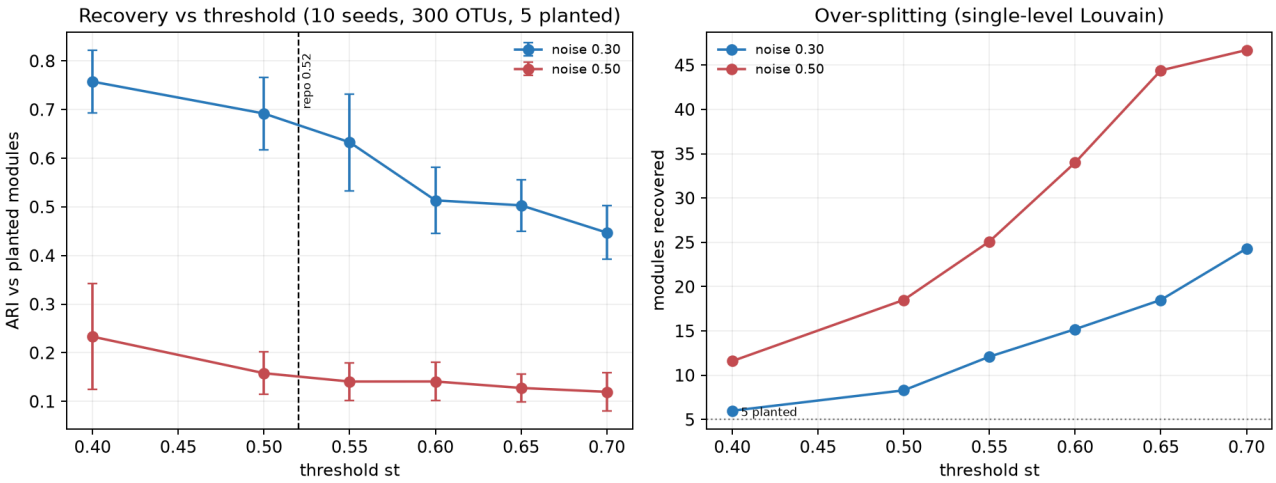


Figure 5: Recovery falls as the threshold rises, while the RMT procedure scans upward. Right panel: the single-level optimiser over-splits — 5 planted modules become 24 at st = 0.70.

st	ARI (noise 0.30)	modules	ARI (noise 0.50)	modules
0.40	0.758 ± 0.065	6.0	0.233 ± 0.109	11.6
0.50	0.692 ± 0.074	8.3	0.158 ± 0.043	18.5
0.55	0.633 ± 0.100	12.1	0.141 ± 0.038	25.1
0.60	0.513 ± 0.068	15.2	0.141 ± 0.039	34.0

0.70	0.447 ± 0.055	24.3	0.120 ± 0.039	46.7

The consequence is uncomfortable and worth stating plainly: recovery is best at the **lowest** threshold tested, and decays as st rises, whereas the Poisson criterion walks upward. The criterion optimises spectral randomness, not community recovery. At noise 0.50 almost nothing is recoverable at any threshold, so the threshold governs sparsity, not validity.

7 Results on the real data (exploratory)

Fixed threshold 0.52 for comparability with the 2025 work, 999 permutations.

condition	n / E	M	avg CC	GD	modules	Mantel r (p)
box	149 / 694	0.631	0.561	3.22	15	0.173 (0.005)
ox	166 / 836	0.650	0.640	3.77	20	0.125 (0.017)
anox	165 / 850	0.583	0.628	3.63	17	0.077 (0.086)

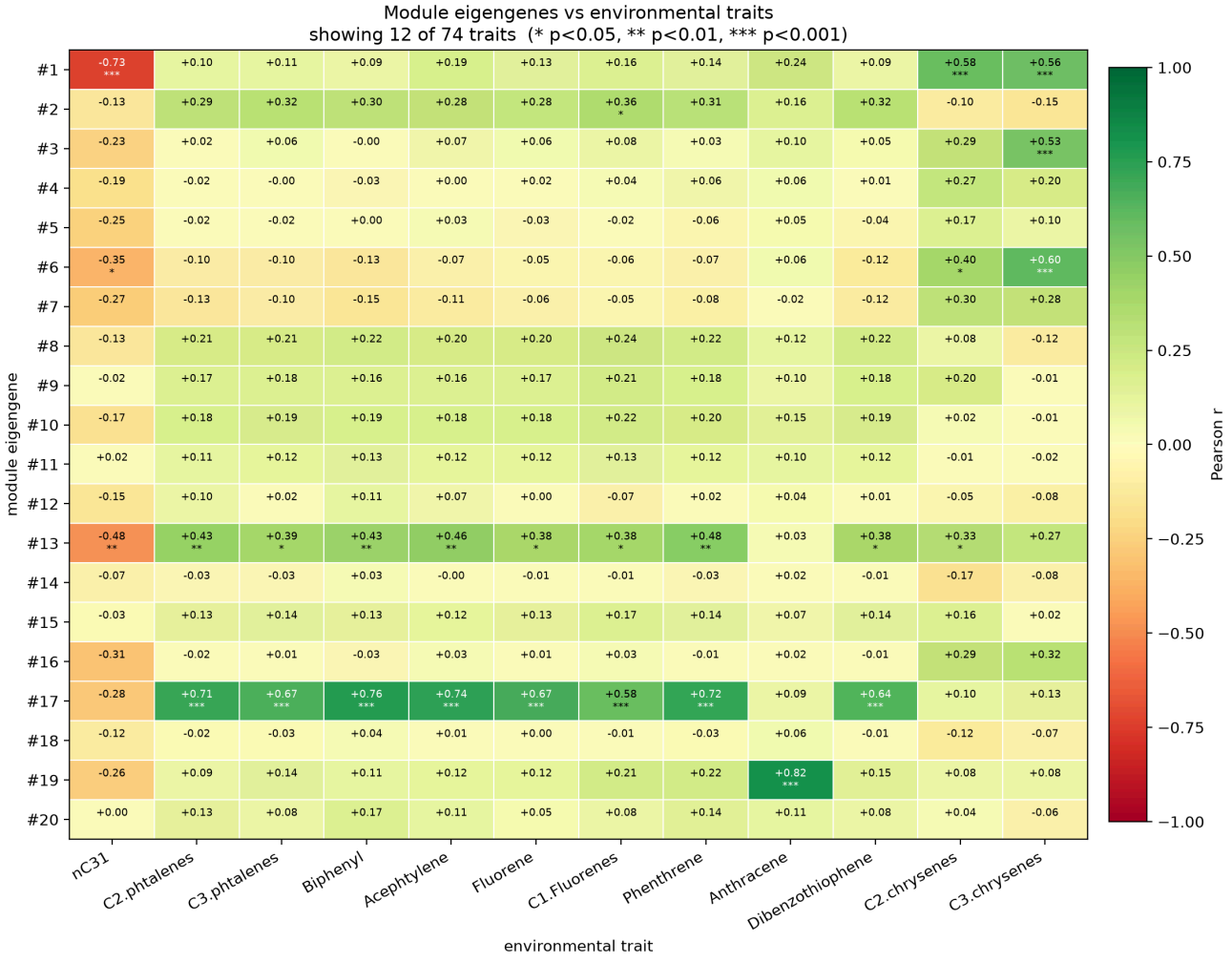


Figure 6: Module eigengene vs trait correlations, ox condition. 20 module rows; the 12 of 74 traits shown are those with any significant correlation — a display filter that must be replaced by q-values or a pre-declared panel.

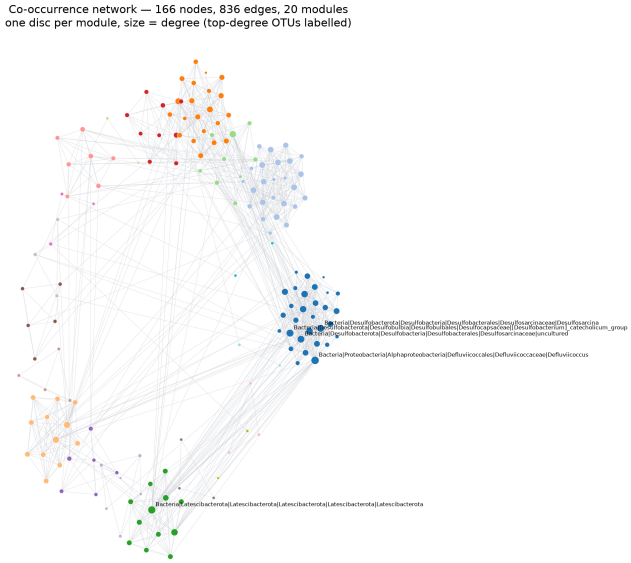


Figure 7: Co-occurrence network (166 nodes, 836 edges). One disc per module; force-directed layouts were unreadable at this density.

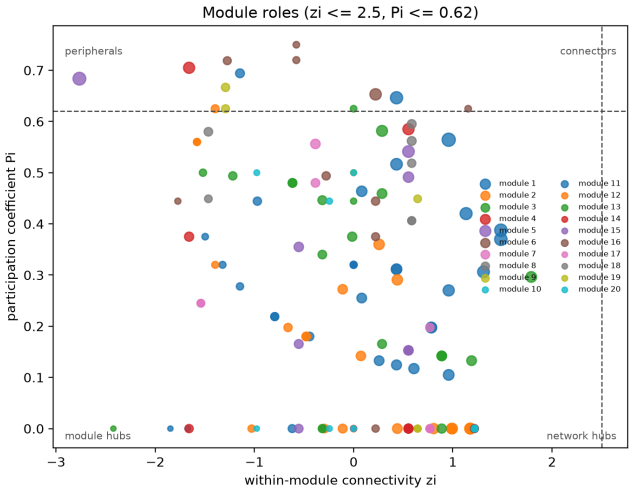


Figure 8: Module roles in the zi–Pi plane. Most OTUs are peripherals, consistent with the paper's 90.9 %.

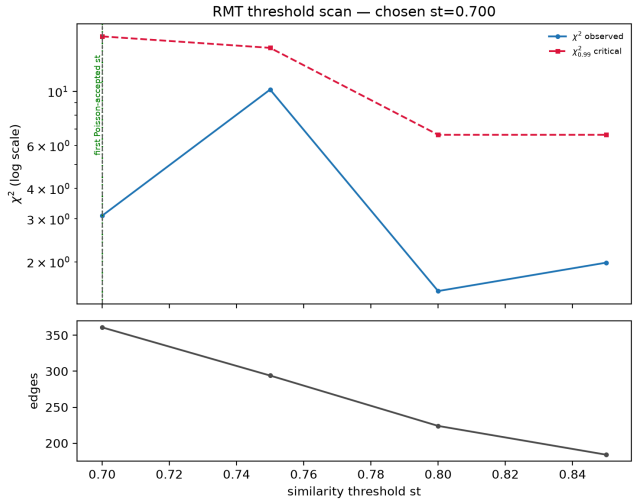


Figure 9: RMT diagnostic. With the faithful weighted matrix and $\alpha = 0.01$ no threshold is estimable on this data.

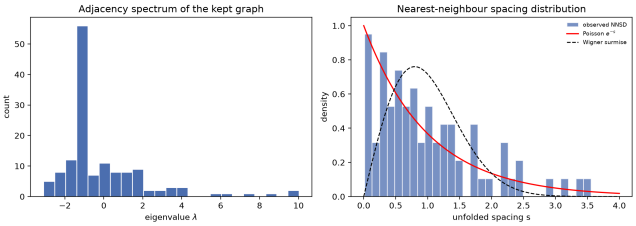


Figure 10: Adjacency spectrum and nearest-neighbour spacing distribution vs Poisson and Wigner.

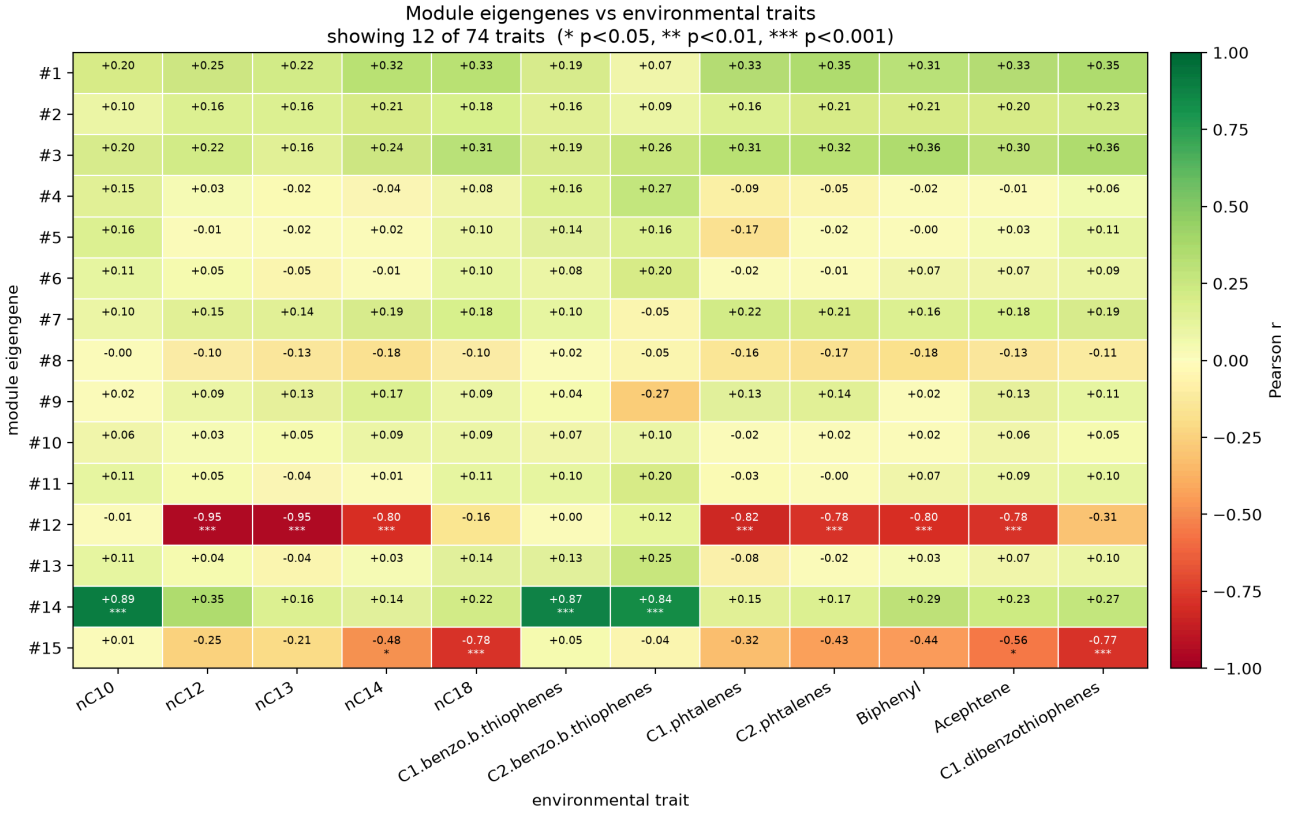


Figure 11: box condition, shown for comparison: fewer modules (15) and the lowest Mantel correlation of the three.

8 What remains

The honest status is that the **software** is in good shape and the **science** is not finished.

1. **Threshold policy**: unresolved. Layer 1 and 2 are specified but not implemented; the calibration says the automatic criterion is anti-correlated with recovery, so publishing an RMT threshold as "the" threshold is not defensible.
2. **Multiple testing**: Benjamini–Hochberg q-values are now computed (`fig5_q.csv`), but the display filter still selects traits by raw significance, which is cherry-picking under multiplicity. Fix pending.
3. **Null model**: M, avg CC and GD still have no null distribution. Without a degree-preserving rewiring null, "M = 0.65 is high" is unquantified.
4. **Over-splitting**: the single-level Louvain optimiser fragments communities (5 planted → 24 recovered). Multilevel aggregation is the fix.
5. **Compositionality**: Pearson on relative abundances is biased; CLR-based similarity is the standard mitigation and is not implemented.
6. **Provenance**: the vault is not under version control and `/real/` is excluded from git, so validation inputs live only on RawStorage.

9 Reproducibility

```
cargo build --release                    # 18 tests
hydro-mena run --abundance real/abundance_ox.csv \
               --traits real/traits_ox.csv \
               --out-dir real/review_ox/s52 --st 0.52 --permutations 999
pixi run python viz.py all real/review_ox/s52 --out real/review_ox/figs
pixi run python gui/app.py --root . --port 8770
```